ACH Geopolitical Forecasting Agent

A Multi-Nation Analysis of Competing Hypotheses Pipeline

CMU Agentic AI Certificate — Capstone Final Report

James Kajdasz • james.kajdasz@gmail.com

June 23, 2026

1. Project Title

ACH Geopolitical Forecasting Agent: A Three-Tier Multi-Agent System for Automated Analysis of Competing Hypotheses Applied to International Relations News

2. Problem and User

Intelligence analysts and policy researchers face an ever-growing torrent of open-source geopolitical news. Manually applying structured analytic techniques such as Analysis of Competing Hypotheses (ACH) to hundreds of articles per week is labor-intensive and prone to cognitive biases—confirmation bias in particular causes analysts to overweight evidence that confirms a pre-existing view and to discount disconfirming evidence.

The intended primary users are intelligence analysts, policy researchers, and national-security students who need a systematic, bias-resistant way to track how open-source evidence about a nation's foreign-policy posture accumulates over time. The system is equally useful as a pedagogical tool for students of structured analytic techniques who want to see ACH applied at scale.

This problem matters because unstructured reading of news feeds is epistemically unreliable: the ACH methodology, formalized by Richards Heuer in Psychology of Intelligence Analysis (1999), addresses this by forcing the analyst to weigh evidence against all competing explanations simultaneously rather than building a case for a single favored conclusion. Automating the evidence-ingestion and scoring steps removes repetitive manual work and makes the technique tractable across many nations in parallel.

3. System Goal and Scope

The system is designed to automatically ingest full-text geopolitical news articles, score each article's diagnostic value against a fixed set of competing hypotheses for each monitored nation, and maintain a persistent, versioned ACH evidence matrix per nation. Success is defined as:

- Consistent retrieval of new, non-duplicate articles from The Guardian Content API for each configured nation.
- Accurate mapping of article content to evidence marks (++, +, N/A, -, --) across multiple LLM passes, with measurable self-consistency confidence.
- A correctly ranked ACH matrix (lowest inconsistency = most likely hypothesis, per Heuer Step 5) that accumulates correctly across runs without data loss.
- A self-contained HTML output per nation plus a summary dashboard that an analyst can inspect in a browser without installing additional software.

Scope boundaries and constraints:

- Evidence comes exclusively from The Guardian Open Platform Content API (full body text, free tier). Reuters was evaluated but excluded due to terms-of-service restrictions on scraping and licensed-only full text.

- The LLM is a locally-hosted open-source model served by Ollama. No external API calls to commercial LLM providers are made, preserving air-gap compatibility and eliminating per-token costs.
- Each nation is assessed on a single analytic axis: whether it supports, is neutral toward, or opposes the United States (h1/h2/h3). More nuanced multi-axis assessment is out of scope.
- The system is a batch pipeline, not a streaming service. It is designed to be run on a schedule (e.g., daily) rather than in real time.
- There is no user chat interface by design, to avoid prompt-injection risk from scraped article content feeding into a conversational loop.

4. Final System Architecture

The system implements a three-tier multi-agent pipeline. The main orchestrator (main.py) loops over each configured nation and sequentially invokes the three agents. Each nation's pipeline is fully isolated—there is no shared article state between nation passes, preventing cross-contamination of evidence.

Tier 1 — ScraperAgent

ScraperAgent queries the Guardian Content API using a nation-specific search query (e.g., "Israel United States relations") and returns full-body ArticleData objects. It deduplicates against a persistent CSV of processed URLs (data/processed_urls.csv) maintained by FileManager, ensuring each article is assessed at most once across all pipeline runs. The agent enforces a domain whitelist and records every scrape attempt to the audit log.

Tier 2 — AssessmentAgent

AssessmentAgent implements comparative ACH scoring. For each new article it runs llm_num_passes (default 10) independent LLM calls at temperature 0.7, presenting all three competing hypotheses together in a single prompt so the model can discriminate between them. The evidence mark for each hypothesis is determined by majority vote across passes; confidence is the agreement fraction (most-common count / total passes). Articles are flagged for human review when any per-hypothesis confidence falls below the configurable threshold (default 0.6).

Tier 3 — MatrixAgent

MatrixAgent maintains the Heuer Chapter 8 evidence matrix: articles as rows, hypotheses as columns. It loads existing state from data/matrix/{nation_id}/matrix_state.json at startup (preserving prior evidence across runs), ingests new AssessmentResult rows (deduplicating by article_id), ranks hypotheses by inconsistency score (lowest = most likely, per Heuer Step 5), and writes three artifacts: the updated JSON state, a stable acch_matrix.html (the analyst's working view), and a timestamped snapshot for the audit trail. Old snapshots are pruned once the directory exceeds matrix_storage_cap_gb (default 1 GB).

Summary Page

After all nations are processed, render_summary_html() produces data/matrix/summary.html—a standalone page with a multi-line Canvas chart showing the cumulative US-alignment score over time for every nation, with legend links that navigate to each nation's individual ACH matrix. The per-article alignment score is computed as weight[h1_mark] − weight[h3_mark] where weights are ++:2, +:1, N/A:0, -:−1, --:−2.

Supporting Tools

LLMInterface (tools/llm_interface.py): Thin HTTP client over Ollama's /api/generate endpoint. Implements the comparative ACH prompt and a multi-strategy evidence-mark parser.

WebScraper (tools/web_scraper.py): Guardian API client with exponential-backoff retry and OS-truststore TLS verification (via the truststore package, required behind corporate TLS-intercepting proxies).

FileManager (tools/file_manager.py): Processed-URL persistence and per-nation snapshot directory management.

AuditLogger (tools/audit_logger.py): Named Python loggers write to logs/agent_interactions.log, logs/assessments.log, and logs/errors.log. A rich console handler provides live terminal output with colored tables.

MatrixView (tools/matrix_view.py): Pure-presentation module that renders the ACH matrix as a color-coded HTML table and embeds an inline Canvas line graph (no external JS dependencies).

5. Design Evolution Across the Program

v1 — Single-Nation Proof of Concept

Version 1 established the basic three-tier pipeline for a single nation (China–US relations) and validated the core loop: scrape → score → accumulate. The LLM prompt evaluated one hypothesis per call, requiring three sequential LLM calls per article. The matrix was single-nation only, rendered as a static HTML table with no line graph. The test suite (86 hermetic tests) was also established in v1, mocking all external I/O (Guardian API, Ollama) so the pipeline could be verified without network or model access.

v2 — Comparative ACH Scoring

The key insight driving v2 was that ACH diagnosticity is fundamentally comparative: evidence is diagnostic only insofar as it distinguishes between competing hypotheses. Scoring one hypothesis in isolation loses this discriminating power. v2 replaced the per-hypothesis prompt with a single comparative prompt presenting all three hypotheses simultaneously, reducing LLM calls per article from 3×passes to 1×passes while improving mark coherence. The evidence mark parser was also hardened to handle the wider variety of model output formats.

v3 — Multi-Nation and Summary Dashboard

v3 extended the pipeline to support arbitrarily many nations via a YAML configuration file (config/hypothesis_config.yaml). Each nation runs its own fully isolated Scraper → Assessment → Matrix pass with no shared state. Key additions:

- Nation-keyed hypothesis configuration — adding a nation requires only a YAML edit, no code changes.
- Per-nation HTML pages with a US-alignment line graph (cumulative evidence score over time).
- summary.html — a multi-nation overview page with a single multi-line chart and click-through navigation.
- MatrixAgent always initializes from prior state before scraping, ensuring previously-accumulated evidence carries forward even on runs where no new articles are found.
- URL tracking in EvidenceRow — enables back-linking from the matrix table to the original Guardian article.
- 10 nations configured at launch: China, Iran, Israel, Qatar, Saudi Arabia, Pakistan, Kuwait, Bahrain, Oman, UAE.

6. Implementation Overview

The system is implemented in Python 3.12 and managed with uv (an ultrafast Rust-based package manager). All runtime dependencies are declared in pyproject.toml.

Language and Runtime

- Python 3.12 — type hints throughout; Pydantic v2 models for inter-agent data contracts.
- uv — dependency management and virtual environment; all commands run via uv run python main.py.
- Ollama — local LLM serving; the pipeline targets llama3.1 (8B) with an 8 192-token context window.

Key Libraries

- pydantic / pydantic-settings — data validation and environment-variable-driven configuration.
- requests — HTTP client for both the Guardian API and the Ollama endpoint.
- truststore — OS trust-store injection for TLS verification behind corporate proxies.
- PyYAML — hypothesis configuration loading.
- rich — colored console output and live hypothesis-ranking tables.
- pytest / pytest-cov — hermetic test suite (86 tests, no network or LLM required).
- python-docx / lxml — used to generate this report.
- langgraph, langchain, torch, sentence-transformers — installed but not used at runtime; retained for potential future extensions.

Configuration

All tunable parameters (LLM endpoint, model, temperature, number of passes, confidence threshold, Guardian API key, article window in days, storage cap) are exposed as Pydantic Settings fields that map case-insensitively to environment variables. A .env file is supported via python-dotenv. The literal "test" Guardian API key works out of the box for development; production deployments should set GUARDIAN_API_KEY to a registered free key.

Data Flow

ArticleData objects (Pydantic) cross the Tier 1 → Tier 2 boundary. AssessmentResult objects (Pydantic) cross the Tier 2 → Tier 3 boundary. MatrixAgentState (dataclass) is the Tier 3 output and is also consumed by the orchestrator to build the summary page. EvidenceRow objects serialize to/from JSON for cross-run persistence. This clean schema contract means each tier can be unit-tested in isolation with typed mock data.

7. Evaluation and Results

Hermetic Unit and Integration Tests

The primary formal evaluation is the hermetic test suite (tests/), which covers all three agents and all supporting tools. All external I/O—Guardian API calls and Ollama HTTP calls—is mocked via pytest fixtures in conftest.py, so the suite runs reliably without network access or a running LLM. 86 tests pass; coverage targets the agents/ and tools/ packages.

Test files and their focus:

- test_scraper_agent.py — URL deduplication, ArticleData construction, audit logging.
- test_assessment_agent.py — self-consistency calculation, multi-pass aggregation, human-review flagging.
- test_matrix_agent.py — evidence accumulation, inconsistency ranking (Heuer Step 5), JSON persistence, HTML rendering, snapshot pruning.
- test_llm_interface.py — evidence-mark parsing across a wide range of model output formats; connection-failure fallback.
- test_web_scraper.py — Guardian API query construction, retry/backoff behavior, date filtering.
- test_matrix_view.py — HTML table rendering, line-graph score computation, summary page structure.
- test_file_manager.py — processed-URL CSV read/write, directory sizing, snapshot cleanup.

End-to-End Pipeline Verification

The full pipeline (uv run python main.py) was run end-to-end with a live Ollama instance (llama3.1:8b) and a registered Guardian API key. The run confirmed:

- Correct per-nation isolation: each of the 10 configured nations produced a separate acch_matrix.html under data/matrix/{nation_id}/.
- Hypothesis ranking consistency: nations expected a priori to be more aligned with the US (e.g., Israel, Kuwait, Bahrain) ranked h1 (supports US) first by inconsistency score after evidence accumulation.
- Cross-run persistence: a second pipeline run correctly loaded prior evidence rows and added only new (non-duplicate) articles.
- Summary page: data/matrix/summary.html rendered correctly in a browser, displaying the multi-line alignment chart with all configured nations.

Known Limitations of Assessment Accuracy

Assessment directional accuracy is bounded by the capability of the local 8B model. Llama 3.1 (8B) performs well on clear, high-signal articles but struggles with:

- Subtlety and irony — sarcastic or hedged diplomatic language may be marked incorrectly.
- Long articles near the 8 192-token context boundary — content beyond the window is silently truncated by Ollama.
- Low-confidence cases — the human-review flag surfaces these explicitly (confidence < 0.6), so they are visible to the analyst rather than silently incorrect.

8. Safety and Reliability Considerations

Human Oversight

The most important safety mechanism is the human-review flag. When any per-hypothesis confidence score falls below the configurable threshold (default 0.6), the article is flagged with flagged_for_human_review=True. Flagged counts are printed to the console at the end of each nation's assessment pass and written to the assessment log. This surfaces uncertainty explicitly rather than silently presenting a low-confidence assessment as authoritative.

Prompt-Injection Defense

There is no user chat interface. All input to the LLM flows through the fixed ACH scoring prompt template, where the article body occupies a clearly delimited NEWS ARTICLE: section. There is no pathway for adversarial article content to alter the prompt structure (no f-string interpolation of hypothesis names or instructions from article content). The output parser looks only for hypothesis-id-prefixed lines, ignoring any other text the model generates.

LLM Fallback

If an individual LLM call fails (network error, timeout, model error), evaluate_hypotheses() catches the RuntimeError and returns N/A for all hypotheses for that pass. This means one failed pass reduces effective sample size by one but does not abort the run. The connection

check at startup (_verify_connection) causes main.py to fail fast if Ollama is unreachable, before any articles are fetched.

TLS and Network Security

WebScraper calls truststore.inject_into_ssl() at initialization (controlled by use_system_truststore=True). This keeps TLS certificate verification enabled while using the OS trust store rather than the certifi bundle—required in corporate environments where a TLS-intercepting proxy has its CA installed in the OS but not in certifi. Certificate verification is never disabled (verify=False is not used anywhere in the codebase).

Data Integrity

Evidence rows are deduplicated by article_id at both the Scraper level (processed_urls.csv prevents re-fetching) and the MatrixAgent level (ingest_assessment replaces an existing row for the same article_id rather than appending a duplicate). JSON state is written atomically: the file is overwritten only after all rows have been serialized. If write fails, the prior state file is still intact.

Storage Management

MatrixAgent.cleanup_old_snapshots() prunes timestamped HTML snapshots (acch_matrix_v*.html) once the nation's matrix directory exceeds matrix_storage_cap_gb (default 1 GB), deleting oldest first. The stable acch_matrix.html (the analyst's current view) and matrix_state.json are never pruned.

Audit Trail

Three separate rotating log files provide an audit trail: agent_interactions.log (all scrape and assessment events), assessments.log (per-article mark distributions and confidence scores), and errors.log (exceptions with full tracebacks). Every LLM assessment result is logged before being returned, enabling post-hoc review of model behavior.

9. Limitations and Next Steps

Current Limitations

- Model capability ceiling: The 8B local model is the binding constraint on assessment quality. Nuanced geopolitical language, implicit signaling, and long-form analysis are handled inconsistently.
- Single news source: Using only The Guardian introduces source-specific framing bias. The Guardian's editorial perspective on US foreign policy may skew evidence marks systematically for certain nations.

- Context window truncation: Articles longer than ~8 192 tokens are silently truncated by Ollama before the model sees them. Long investigative pieces may have their most diagnostic content cut off.
- Hypothesis axis: The h1/h2/h3 supports/neutral/opposes framing is deliberately simple. It cannot capture conditional alignment, sectoral divergence (e.g., economic cooperation alongside military rivalry), or evolving postures within a single article.
- Batch-only operation: The pipeline is a one-shot batch job. There is no incremental or event-driven update mode, and no scheduling built in.
- No retrieval-augmented context: The LLM has no access to background knowledge about ongoing events beyond what is in the article itself. Historical context that an analyst would bring is absent.

Realistic Next Steps

- Larger local model: Upgrade to llama3.1:70b (if VRAM allows) or a quantized 34B model. The dual TITAN RTX box has 48 GB of VRAM, making Q4 quantization of a 34B model feasible.
- Additional news sources: Add Reuters via an official licensed feed, AP News, BBC, or Al Jazeera. A source-weighting scheme in the matrix would let the analyst assign higher credibility to primary sources.
- Retrieval-augmented assessment: Inject a country-specific background context block (retrieved from a local vector store) into the ACH prompt, giving the model the historical baseline an analyst would bring.
- Scheduled execution: Add a cron-compatible entry point or integrate with Windows Task Scheduler / systemd to run the pipeline daily and auto-publish the HTML outputs.
- Extended hypothesis axis: Add a second axis (e.g., economic alignment, military cooperation) so each nation is scored on a 2-D grid rather than a single supports/neutral/opposes dimension.
- Analyst feedback loop: Provide a lightweight web UI where an analyst can override a flagged assessment mark; overrides are recorded as ground truth and used to fine-tune the prompt or evaluate model accuracy.

10. Public GitHub Repository

The project repository is available at:

https://github.com/jkajdasz/ach-geopolitical-forecasting-agent

The repository includes:

- README.md — project overview, problem statement, architecture diagram, setup steps (uv sync, Ollama install, Guardian API key), and usage instructions.
- agents/ — ScraperAgent, AssessmentAgent, MatrixAgent, and base schemas.
- tools/ — LLMInterface, WebScraper, FileManager, AuditLogger, MatrixView.

- config/ — Settings, hypothesis_config.yaml (10 nations), domain_whitelist.txt.
- tests/ — 86 hermetic tests with mocked I/O.
- documentation/ — Reuters delivery overview (context for source selection decision), Psychology of Intelligence Analysis (Heuer, 1999), and this report.
- pyproject.toml — all dependencies and tool configuration.
- .env.template — environment variable reference.

A technical reader can reproduce the pipeline by: (1) cloning the repository, (2) running uv sync, (3) starting Ollama with ollama pull llama3.1, and (4) running uv run python main.py. The test suite (uv run pytest) requires no external services.

10. Unlisted Presentation Video

A video presentation of the final video is available at:

https://youtu.be/ijaKZi1KcTU

- The video is unlisted and is not searchable. It may be viewed by anyone who has the link.